

Relatorio Tecnico - TP01: Agentes de Busca em Ambientes de Labirinto

1. Introducao

Este trabalho pratico (TP01) consiste na implementacao de um sistema de agentes inteligentes capazes de resolver problemas de busca em ambientes de labirinto. O projeto reutiliza e estende componentes desenvolvidos em atividades anteriores (ATV02), adaptando-os para suportar novos requisitos e padroes de arquitetura mais robustos.

2. Heranca deCodigo da ATV02

2.1 Componentes Reutilizados

O TP01 aproveita os seguintes componentes principais da ATV02:

2.1.1 Modulo `exibir.py`

- * Funcionalidade: Renderizacao visual do labirinto e exibicao de metricas de busca
- * Estrutura mantida:
 - * `imprimir_labirinto()`: Funcao que exhibe o labirinto com representacoes visuais
 - * `imprimir_metricas()`: Funcao que imprime estatisticas de execucao
- * Adaptacoes realizadas:
 - * Suporte a estados estendidos (estados com coletas): `LabirintoComColetas` em TP01 pode ter tuplas como `((i, j), frozenset_coletas)`
 - * Funcao auxiliar `_pos()` para extrair posicoes de estados complexos
 - * Novo simbolo 'C' para representar pontos de coleta

2.1.2 Estrutura de Dados `labirinto.py` / `buscas.py`

- * Conceitos mantidos:
 - * Classe `No`: No da arvore/grao de busca com atributos `estado`, `pai`, `acao`, `g`
 - * Classe `ResultadoBusca`: Dataclass que encapsula resultados com metricas
 - * Classe `LabirintoBusca`: Carregamento e representacao do labirinto
- * Evolucoes em TP01:
 - * Introducao de Protocolo Labirinto para polimorfismo
 - * Extensao `LabirintoComColetas` para problemas com multiplos objetivos
 - * Novos atributos em `ResultadoBusca`: `custo_total`, `tempo_ms`, `fronteira_max`

2.1.3 Algoritmos de Busca Classica

Os cinco algoritmos fundamentais foram adaptados e reorganizados:

Originais em ATV02 (metodos da classe `LabirintoBusca`):

- * busca_largura() -> BFS
- * busca_profundidade() -> DFS
- * busca_custo_uniforme() -> UCS
- * busca_gulosa() -> Greedy Best-First
- * busca_weighted_astar() -> Weighted A*

Reorganizados em TP01 (funcoes independentes em buscas/classicas/):

- * bfs.py - Breadth-First Search
- * dfs.py - Depth-First Search
- * ucs.py - Uniform Cost Search
- * gulosa.py - Greedy Best-First Search
- * a_estrela.py - A* Search

2.2 Padrao de Busca Mantido

A logica fundamental dos algoritmos permanece essencialmente a mesma:

1. Inicializar fronteira com estado inicial
2. Manter registro de estados explorados
3. Enquanto fronteira não estiver vazia:
 - a. Extrair nó da fronteira (estratégia varia por algoritmo)
 - b. Se nó é objetivo, retornar solução
 - c. Marcar como explorado
 - d. Expandir vizinhos e adicionar à fronteira
4. Se fronteira vazia, retornar falha

Diferencas de estrategia por algoritmo:

- * BFS: Fronteira como fila (FIFO) - exploracao por niveis
- * DFS: Fronteira como pilha (LIFO) - exploracao em profundidade
- * UCS: Fronteira como fila de prioridade por custo acumulado
- * Gulosa: Fronteira ordenada por heuristica (distancia Manhattan)
- * A*: Fronteira ordenada por $f(n) = g(n) + h(n)$

3. Adaptacoes para TP01

3.1 Arquitetura Modular

Mudanca significativa: Os algoritmos foram convertidos de metodos de classe para funcoes independentes.

```

ATV02:
lab = LabirintoBusca('arquivo.txt')
resultado = lab.busca_largura()

TP01:
from buscas.classicas.bfs import bfs
from labirinto import LabirintoBusca
lab = LabirintoBusca('arquivo.txt')
resultado = bfs(lab, historico_estados=[])

```

Benefícios:

- * Separacao de responsabilidades
- * Facilita testes unitarios
- * Permite composicao de algoritmos
- * Suporta diferentes tipos de problemas (coletas, restricoes, etc.)

3.2 Suporte a Problemas Estendidos

ATV02: Focava apenas em problemas de ponto A -> ponto B

TP01: Estende para:

- * Problemas com multiplos pontos de coleta
- * Estados complexos: ((posicao_i, posicao_j), coletas_realizadas)
- * Classe LabirintoComColetas que herda de LabirintoBusca
- * Metodos vizinhos() e h() polimorficos

3.3 Metricas Expandidas

ATV02:

```
- algoritmo
- encontrado
- caminho
- acoes
- nos_explorados
- nos_expandidos
- estados_explorados
```

TP01 - adicionado:

```
+ custo_total (custo real do caminho)
+ tempo_ms (tempo de execucao em milissegundos)
+ fronteira_max (tamanho máximo da fronteira durante busca)
```

3.4 Interface Protocol

Novo em TP01: Uso de Protocol para definir contratos sem heranca nominal:

```
@runtime_checkable
class Labirinto(Protocol):
    inicio: Any
    objetivo: Any
    def vizinhos(self, estado: Any) -> List[Tuple]: ...
    def h(self, estado: Any) -> float: ...
    def reconstruir(self, no: 'No') -> Tuple[List, List]: ...
```

Permite que tanto LabirintoBusca quanto LabirintoComColetas sejam usados pelos algoritmos sem heranca explicita.

4. Estrutura de Diretorios

tp01/

```

+-- src/
|   +-- main.py                # Menu interativo
|   +-- labirinto.py           # Estruturas do problema
|   +-- exibir.py              # Renderização (adaptado de ATV02)
|   +-- visualizacao.py        # Geração de HTML
|   +-- gerar_html.py          # Suporte de visualização
|   +-- formulacao_formal.py   # Definição PEAS/formulação
|   +-- agente.py              # Implementação do agente
|   +-- experimentos.py        # Suite de testes
|   +-- files.py               # Utilitários de arquivo
|   +-- buscas/
|       |   +-- classicas/
|       |       |   +-- bfs.py          # (adaptado de ATV02)
|       |       |   +-- dfs.py          # (adaptado de ATV02)
|       |       |   +-- ucs.py          # (adaptado de ATV02)
|       |       |   +-- gulosa.py        # (adaptado de ATV02)
|       |       |   +-- a_estrela.py     # (adaptado de ATV02)
|       |       +-- local/
|       |           |   +-- hill-climbing.py # Hill Climbing
|       |           |   +-- simulated_annealing.py # Simulated Annealing
|       |           +-- online/
|       |               +-- # Algoritmos online (futuro)
|   +-- mapas/
|       +-- lab1.txt            # Labirinto simples
|       +-- lab2.txt            # Com coletas
|       +-- lab3.txt            # Serpentino
|       +-- lab4.txt            # Complexo
+-- peas.md                    # Formulação PEAS
+-- formulacao_formal.md       # Especificação formal

```

5. Algoritmos de Busca Classica

5.1 BFS (Breadth-First Search)

Origem: Adaptado de `LabirintoBusca.busca_largura()` em ATV02

Características:

- * Exploracao por niveis (nivel 0, 1, 2, ...)
- * Fronteira: fila (FIFO)
- * Completo: Sim (encontra solucao se existir)
- * Otimo: Sim (para custos uniformes)

Complexidade:

- * Tempo: $O(V + E)$
- * Espaco: $O(V)$

5.2 DFS (Depth-First Search)

Origem: Adaptado de `LabirintoBusca.busca_profundidade()` em ATV02

Características:

- * Exploracao em profundidade maxima primeiro
- * Fronteira: pilha (LIFO)
- * Completo: Nao (pode entrar em ciclo infinito)
- * Otimo: Nao

Complexidade:

- * Tempo: $O(V + E)$
- * Espaco: $O(V)$

5.3 UCS (Uniform Cost Search)

Origem: Adaptado de LabirintoBusca.busca_custo_uniforme() em ATV02

Caracteristicas:

- * Expande no com menor custo acumulado $g(n)$
- * Fronteira: fila de prioridade (min-heap)
- * Completo: Sim
- * Otimo: Sim (encontra caminho de menor custo)

Implementacao:

- * Custo acumulado armazenado em $No.g$
- * Usa heapq para eficiencia

5.4 Busca Gulosa (Greedy Best-First)

Origem: Adaptado de LabirintoBusca.busca_gulosa() em ATV02

Caracteristicas:

- * Expande no com menor heuristica $h(n)$
- * Fronteira: fila de prioridade por $h(n)$
- * Completo: Nao (depende da heuristica)
- * Otimo: Nao

Heuristica usada: Distancia Manhattan

$$h(n) = |x_{\text{atual}} - x_{\text{objetivo}}| + |y_{\text{atual}} - y_{\text{objetivo}}|$$

5.5 A* (A-Star)

Origem: Adaptado de LabirintoBusca.busca_weighted_astar() em ATV02

Caracteristicas:

- * Expande no com menor $f(n) = g(n) + h(n)$
- * Combina custo real e heuristica
- * Fronteira: fila de prioridade por $f(n)$
- * Completo: Sim (se h e admissivel)
- * Otimo: Sim (se h e admissivel)

Implementacao em TP01:

```
f = g + h(estado)
heapq.heappush(frenteira, (f, No(estado, g=g)))
```

6. Diferencas Tecnicas Principais

Aspecto	ATV02	TP01
Estrutura Algoritmos	Metodos de classe	Funcoes independentes
Tipos de Problemas	Ponto A -> B	A -> B e com coletas
Estados	Tupla simples (i,j)	Complexos: ((i,j), frozenset)
Interface	Classe concreta	Protocol (typing)
Metricas	7 atributos	10 atributos
Menu	Via input() interativo	Menu estruturado + experimentos
Saida	Console	Console + HTML

7. Fluxo de Execucao Tipico

```
1. Usuario executa: python main.py

2. Menu oferece opcoes:
- Escolher labirinto (1-4 ou custom)
- Executar busca classica
- Executar busca local
- Ver experimentos
- Gerar visualizacao HTML

3. Para busca classica:
- Carregar mapa com LabirintoBusca()
- Selecionar algoritmo (BFS, DFS, UCS, etc.)
- Chamar funcao do algoritmo: bfs(lab, [])
- Exibir resultado com imprimir_labirinto()
- Mostrar metricas com imprimir_metricas()

4. Para busca com coletas:
- Carregar mapa
- Envolver em LabirintoComColetas()
- Usar mesmo algoritmo: bfs(lab_coletas, [])
- Estados automaticamente incluem coletas
```

8. Conclusao

O TP01 demonstra evolucao arquitetural mantendo nucleo funcional estavel:

- v Reutilizacao eficiente de codigo ATV02
- v Modularizacao atraves de separacao de algoritmos
- v Extensibilidade via Protocols e heranca
- v Manutenibilidade com separacao de responsabilidades
- v Adaptacao para problemas mais complexos

Os algoritmos fundamentais de busca mantem sua essencia enquanto a arquitetura evoluiu para suportar novos requisitos de forma elegante e extensivel.

Documentado em: 25 de maio de 2026

Componentes de ATV02 utilizados: exhibir.py, estrutura LabirintoBusca, 5 algoritmos classicos

Extensoes do TP01: LabirintoComColetas, Protocol Labirinto, buscas locais, visualizacao HTML